

操作系统实验报告

姓名：李宗杭 学号：2311451 学院：软件学院

实验日期：2025.11.19 星期三 11-12 节

实验题目：目录复制（多线程）

一、实验内容

用 C/C++ 实现多线程目录拷贝，并与单进程、多进程、系统 cp 命令进行对比。

二、实现步骤

1、前置准备

(1) 安装/更新核心编译工具

```
sudo apt-get install -y build-essential
```

(2) 下载并解压最新 Linux 内核用作测试目录

```
wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.16.12.tar.xz
```

```
tar -Jxvf linux-6.16.12.tar.xz
```

2、代码编写

(1) 头文件引入与全局变量

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <dirent.h>
6  #include <sys/stat.h>
7  #include <sys/types.h>
8  #include <fcntl.h>
9  #include <errno.h>
10 #include <pthread.h>
11
12 // ===== 全局变量 =====
13
14 #define MAX_QUEUE_SIZE 1024 // 待拷贝文件队列最大长度
15 #define MAX_THREADS 4      // 消费者线程数
16 #define MAX_PATH 1024      // 路径最大长度
17
18 // 待拷贝文件的队列节点
19 typedef struct FileNode {
20     char src_path[MAX_PATH];
21     char dest_path[MAX_PATH];
22     struct FileNode* next;
23 } FileNode;
24
25 // 队列结构体
26 typedef struct FileQueue {
27     FileNode* head;
28     FileNode* tail;
29     int size;
30 } FileQueue;
31
32 FileQueue g_queue; // 全局待拷贝队列
33 pthread_mutex_t g_mutex; // 互斥锁
34 int g_is_queue_closed = 0; // 生产者遍历结束标记（队列是否关闭）
35
```

(2) 工具函数：队列初始化

```
36 // ===== 工具函数 =====
37
38 /**
39  * 队列初始化：清空队列
40  */
41 void queue_init() {
42     g_queue.head = NULL;
43     g_queue.tail = NULL;
44     g_queue.size = 0;
45 }
```

(3) 工具函数：入队操作

```
47  /**
48   * 入队操作
49   * 将文件路径加入队列
50   * 参数: src_path (源文件路径), dest_path (目标文件路径)
51   * 返回: 0 (成功), -1 (队列满)
52   */
53  int queue_push(const char* src_path, const char* dest_path) {
54      pthread_mutex_lock(&g_mutex); // 加锁, 保护临界资源
55
56      // 队列满则返回失败
57      if (g_queue.size >= MAX_QUEUE_SIZE) {
58          pthread_mutex_unlock(&g_mutex);
59          return -1;
60      }
61
62      // 创建新节点
63      FileNode* new_node = (FileNode*)malloc(sizeof(FileNode));
64      if (new_node == NULL) {
65          perror("malloc node failed");
66          pthread_mutex_unlock(&g_mutex);
67          return -1;
68      }
69      strcpy(new_node->src_path, src_path);
70      strcpy(new_node->dest_path, dest_path);
71      new_node->next = NULL;
72
73      // 加入队列尾部
74      if (g_queue.tail == NULL) { // 队列为空
75          g_queue.head = new_node;
76          g_queue.tail = new_node;
77      } else {
78          g_queue.tail->next = new_node;
79          g_queue.tail = new_node;
80      }
81      g_queue.size++;
82
83      pthread_mutex_unlock(&g_mutex); // 解锁
84      return 0;
85  }
```

(4) 工具函数：出队操作：

```
87  /**
88   * 出队操作
89   * 从队列取出文件路径
90   * 参数: src_path (源文件路径缓冲区), dest_path (目标文件路径缓冲区)
91   * 返回: 0 (成功), -1 (队列空)
92   */
93  int queue_pop(char* src_path, char* dest_path) {
94      pthread_mutex_lock(&g_mutex);
95
96      // 队列空则返回失败
97      if (g_queue.size == 0) {
98          pthread_mutex_unlock(&g_mutex);
99          return -1;
100     }
101
102     // 取出队首节点
103     FileNode* temp = g_queue.head;
104     strcpy(src_path, temp->src_path);
105     strcpy(dest_path, temp->dest_path);
106
107     // 更新队列头指针
108     g_queue.head = temp->next;
109     if (g_queue.head == NULL) { // 队列变空
110         g_queue.tail = NULL;
111     }
112     g_queue.size--;
113
114     // 释放节点内存
115     free(temp);
116     pthread_mutex_unlock(&g_mutex);
117     return 0;
118 }
```

(5) 工具函数：路径拼接

```
120  /**
121   * 路径拼接函数
122   * 源路径 + 文件名 → 目标路径
123   * 参数: dest (目标路径缓冲区), src_dir (源目录路径), filename (文件名)
124   */
125  void path_join(char* dest, const char* src_dir, const char* filename) {
126      strcpy(dest, src_dir);
127      // 源路径末尾没有 '/', 则补充
128      if (dest[strlen(dest)-1] != '/') {
129          strcat(dest, "/");
130      }
131      strcat(dest, filename);
132  }
```

(6) 工具函数：单个文件拷贝

```
134  /**
135   * 文件拷贝函数
136   * 拷贝单个文件，保留权限
137   * 参数: src_file (源文件路径), dest_file (目标文件路径)
138   * 返回: 0 (成功), -1 (失败)
139   */
140  int copy_file(const char* src_file, const char* dest_file) {
141      // 打开源文件
142      int src_fd = open(src_file, O_RDONLY);
143      if (src_fd == -1) {
144          perror("open src file failed");
145          return -1;
146      }
147
148      // 获取源文件状态 (权限、大小)
149      struct stat src_stat;
150      if (lstat(src_file, &src_stat) == -1) {
151          perror("lstat src file failed");
152          close(src_fd);
153          return -1;
154      }
155
156      // 创建目标文件，保留源文件权限
157      int dest_fd = open(dest_file, O_WRONLY | O_CREAT | O_TRUNC, src_stat.st_mode);
158      if (dest_fd == -1) {
159          perror("open dest file failed");
160          close(src_fd);
161          return -1;
162      }
163
164      // 循环读写 (4KB缓冲区)
165      char buf[4096];
166      ssize_t read_len, write_len; // ssize_t支持负数 (表示错误)
167      while ((read_len = read(src_fd, buf, sizeof(buf))) > 0) {
168          write_len = write(dest_fd, buf, read_len);
169          if (write_len != read_len) { // 写入字节数不匹配
170              perror("write file failed");
171              close(src_fd);
172              close(dest_fd);
173              return -1;
174          }
175      }
176
177      // 检查读操作是否出错
178      if (read_len == -1) {
179          perror("read file failed");
180          close(src_fd);
181          close(dest_fd);
182          return -1;
183      }
184
185      // 关闭文件描述符，释放资源
186      close(src_fd);
187      close(dest_fd);
188      return 0;
189  }
```

(7) 工具函数：遍历目录

```
191  /**
192  * 递归遍历目录
193  * 参数: src_dir (源目录路径), dest_dir (目标目录路径)
194  * 返回: 0 (成功), -1 (失败)
195  */
196  int traverse_dir(const char* src_dir, const char* dest_dir) {
197      // 获取源目录状态 (用于复制权限)
198      struct stat src_stat;
199      if (lstat(src_dir, &src_stat) == -1) {
200          perror("lstat src dir failed");
201          return -1;
202      }
203
204      // 创建目标目录 (允许已存在, 忽略EEXIST错误)
205      if (mkdir(dest_dir, src_stat.st_mode) == -1 && errno != EEXIST) {
206          perror("mkdir dest dir failed");
207          return -1;
208      }
209
210      // 打开源目录
211      DIR* dir = opendir(src_dir);
212      if (dir == NULL) {
213          perror("opendir src dir failed");
214          return -1;
215      }
216
217      // 遍历目录项
218      struct dirent* dir_entry;
219      char src_path[MAX_PATH], dest_path[MAX_PATH];
220      while ((dir_entry = readdir(dir)) != NULL && !g_is_queue_closed) {
221          // 跳过当前目录 (.) 和父目录 (..), 避免循环遍历
222          if (strcmp(dir_entry->d_name, ".") == 0 || strcmp(dir_entry->d_name, "..") == 0) {
223              continue;
224          }
225
226          // 拼接完整路径
227          path_join(src_path, src_dir, dir_entry->d_name);
228          path_join(dest_path, dest_dir, dir_entry->d_name);
229
230          // 根据文件类型处理
231          switch (dir_entry->d_type) {
232              case DT_REG: // 普通文件: 加入队列, 由消费者线程拷贝
233                  while (queue_push(src_path, dest_path) == -1) {
234                      // 队列满时, 短暂休眠后重试, 释放CPU
235                      usleep(500);
236                  }
237                  break;
238              case DT_DIR: // 子目录: 递归遍历, 生产者继续处理
239                  if (traverse_dir(src_path, dest_path) == -1) {
240                      closedir(dir);
241                      return -1;
242                  }
243                  break;
244              case DT_LNK: // 符号链接: 直接复制
245                  if (link(src_path, dest_path) == -1) {
246                      perror("create symlink failed");
247                      closedir(dir);
248                      return -1;
249                  }
250                  break;
251              default: // 其他文件类型暂不处理
252                  printf("unsupported file type: %s\n", src_path);
253                  break;
254          }
255      }
256
257      closedir(dir);
258      return 0;
259  }
```

(8) 线程函数：生产者

```
261 // ===== 线程函数 =====
262
263 /**
264  * 生产者线程函数
265  * 遍历目录，将普通文件加入队列
266  * 参数: void* args (传入源目录和目标目录的指针数组)
267  */
268 void* producer_thread(void* args) {
269     char** dirs = (char**)args;
270     const char* src_dir = dirs[0];
271     const char* dest_dir = dirs[1];
272
273     // 开始遍历目录
274     if (traverse_dir(src_dir, dest_dir) == -1) {
275         perror("producer traverse dir failed");
276         pthread_exit(NULL);
277     }
278
279     // 遍历结束，标记队列关闭
280     pthread_mutex_lock(&g_mutex);
281     g_is_queue_closed = 1;
282     pthread_mutex_unlock(&g_mutex);
283
284     printf("producer thread finished\n");
285     pthread_exit(NULL);
286 }
```

(9) 线程函数：消费者

```
288 /**
289  * 消费者线程函数
290  * 队列取文件，执行拷贝
291  * 参数: void* args (NULL)
292  */
293 void* consumer_thread(void* args) {
294     char src_path[MAX_PATH], dest_path[MAX_PATH];
295
296     while (1) {
297         pthread_mutex_lock(&g_mutex);
298         // 退出条件: 队列空 + 生产者已标记关闭
299         if (g_queue.size == 0 && g_is_queue_closed == 1) {
300             pthread_mutex_unlock(&g_mutex);
301             break;
302         }
303         pthread_mutex_unlock(&g_mutex);
304
305         // 从队列取文件，拷贝
306         if (queue_pop(src_path, dest_path) == 0) {
307             if (copy_file(src_path, dest_path) == -1) {
308                 printf("consumer copy file failed: %s\n", src_path);
309             }
310         } else {
311             // 队列空时，短暂休眠，释放CPU
312             usleep(500);
313         }
314     }
315
316     printf("consumer thread finished\n");
317     pthread_exit(NULL);
318 }
```

(10) 主函数

```
320 // ===== 主函数 =====
321 int main(int argc, char* argv[]) {
322     // 检查命令行参数，必须传入源目录和目标目录
323     if (argc != 3) {
324         printf("用法: %s <源目录> <目标目录>\n", argv[0]);
325         return 1;
326     }
327     const char* src_dir = argv[1];
328     const char* dest_dir = argv[2];
329
330     // 初始化队列和互斥锁
331     queue_init();
332     if (pthread_mutex_init(&g_mutex, NULL) != 0) {
333         perror("pthread_mutex_init failed");
334         return 1;
335     }
336
337     // 创建线程: 1个生产者 + MAX_THREADS个消费者
338     pthread_t producer_tid;
339     pthread_t consumer_tids[MAX_THREADS];
340     char* dirs[] = {(char*)src_dir, (char*)dest_dir}; // 生产者参数
341
342     // 创建生产者线程
343     if (pthread_create(&producer_tid, NULL, producer_thread, (void*)dirs) != 0) {
344         perror("create producer thread failed");
345         pthread_mutex_destroy(&g_mutex);
346         return 1;
347     }
348
349     // 创建消费者线程
350     for (int i = 0; i < MAX_THREADS; i++) {
351         if (pthread_create(&consumer_tids[i], NULL, consumer_thread, NULL) != 0) {
352             perror("create consumer thread failed");
353             // 销毁已创建的线程，标记队列关闭，通知其他线程退出
354             g_is_queue_closed = 1;
355             pthread_join(producer_tid, NULL);
356             for (int j = 0; j < i; j++) {
357                 pthread_join(consumer_tids[j], NULL);
358             }
359             pthread_mutex_destroy(&g_mutex);
360             return 1;
361         }
362     }
363
364     // 等待所有线程结束（主线程阻塞，直到子线程完成）
365     printf("start copying %s to %s...\n", src_dir, dest_dir);
366     pthread_join(producer_tid, NULL);
367     for (int i = 0; i < MAX_THREADS; i++) {
368         pthread_join(consumer_tids[i], NULL);
369     }
370
371     // 销毁互斥锁
372     pthread_mutex_destroy(&g_mutex);
373
374     printf("copy completed successfully!\n");
375     return 0;
376 }
```


3、编译运行

```
gcc mycopy_multi_thread.c -o mycopy_multi_thread -lpthread
```

```
time sudo ./mycopy_multi_thread linux-6.16.12 linux-6.16.12bak_thread
```

4、结果验证与性能对比

如图所示，diff 命令结果无输出，说明拷贝结果正确。

```
LZH2311451@LZH2311451-pc:~/lab06$ gcc mycopy_multi_thread.c -o mycopy_multi_thread -lpthread
LZH2311451@LZH2311451-pc:~/lab06$ time sudo ./mycopy_multi_thread linux-6.16.12 linux-6.16.12bak_thread
输入密码
start copying linux-6.16.12 to linux-6.16.12bak_thread...
producer thread finished
consumer thread finished
consumer thread finished
consumer thread finished
consumer thread finished
copy completed successfully!

real    1m5.039s
user    0m3.590s
sys     1m14.502s
LZH2311451@LZH2311451-pc:~/lab06$ diff -r linux-6.16.12 linux-6.16.12bak_thread
LZH2311451@LZH2311451-pc:~/lab06$
```

与单进程、多进程和系统 cp 命令做对比，其中单进程和多进程直接复用第 2 次实验结果，进程数与线程数都为 4。运行时间：多线程 > 多进程 > 单进程 > 系统命令。多线程的线程切换开销远小于多进程的进程切换，因此效率更高。但是系统命令却比单进程还慢，本以为其会有内核级优化最快，查阅相关信息后，了解到系统 cp 命令是单线程，并且可能会有一些额外的系统操作，因此十分慢。

```
LZH2311451@LZH2311451-pc:~/lab06$ time cp -a linux-6.16.12 linux-6.16.12bak_cp
real    4m11.663s
user    0m5.802s
sys     1m18.300s
```

```
Asever@Asever-pc:~/0$ time ./mycopy linux-6.16.12 linux-6.16.12bak
start copying linux-6.16.12 to linux-6.16.12bak...
copy completed successfully

real    3m30.067s
user    0m2.570s
sys     1m9.057s
Asever@Asever-pc:~/0$ time ./mycopy_multi linux-6.16.12 linux-6.16.12bak_multi
start copying linux-6.16.12 to linux-6.16.12bak_multi...
copy completed successfully

real    1m15.792s
user    0m6.090s
sys     1m58.838s
Asever@Asever-pc:~/0$ diff -r linux-6.16.12 linux-6.16.12bak
Asever@Asever-pc:~/0$
```

三、结论

本次实验成功实现了多线程目录拷贝功能，通过生产者-消费者模型实现了目录遍历与文件拷贝的并行处理，借助互斥锁保证了临界资源的安全访问。经 diff 命令验证，多线程拷贝的目录与源目录内容完全一致，拷贝结果正确。性能对比结果显示，在相同任务量下，多线程拷贝效率优于多进程和单进程。